Adaptive-Mixture Augmented Policy Gradients for Expected-Length-Constrained Reinforcement Learning with Verifiable Rewards

Anonymous Authors[1]

Abstract

Reinforcement learning with verifiable rewards spends most of its compute on fresh language-model trajectories, while ordinary replay is statistically problematic because sequence likelihood ratios become unstable and previously consumed samples depend on the current policy. We study a narrower alternative: at each frozen policy, combine fresh on-policy trajectories with newly generated, single-use trajectories from a known stale actor and an adaptive prompt distribution. A current-policy mixture component bounds the exact joint importance ratio, and frozen reward and length models supply an augmented control variate. Under explicit sampling and decoding assumptions, the resulting one-update gradient estimator is conditionally unbiased. A Lagrange multiplier constrains expected response length instead of silently normalizing the objective per token. This methods-first draft gives the estimator audit and matched-budget training protocol, but reports no performance gain before those experiments are run.

1. Introduction

Language-model reinforcement learning with verifiable rewards (RLVR) can train reasoning behavior from executable tests or exact answer checks. Its simplicity does not remove the cost of generation: a policy update may require many long trajectories, most of which are discarded after one use. Critic-free descendants of REINFORCE (Williams, 1992), such as RLOO (Ahmadian et al., 2024) and GRPO (Shao et al., 2024) reduce value-model overhead, while large-scale systems add clipping, prompt filtering, and token-level aggregation (Yu et al., 2025). These choices improve engineering behavior but change which prompt, rollout, or token population receives weight.

Reusing trajectories appears attractive, yet "off-policy" covers distinct statistical settings. A replay-buffer trajectory may have selected itself into the buffer through its reward and may already have changed the current policy. Recomputing its likelihood does not restore independence. Sequence-level importance ratios can have extreme variance; token-level ratios do not represent the sampling probability of a sequence-rewarded trajectory. Practical off-policy methods therefore use tapering, buffers, or adaptive clipping (Le Roux et al., 2025; Wan et al., 2026; Xi et al., 2026), knowingly trading bias for stability.

We ask whether a controlled source of stale data can improve gradient efficiency without making an inaccurate replay-unbiasedness claim. At iteration t, a live stale actor produces *new*, never-consumed trajectories from a logged policy and prompt distribution. These are combined with a positive fraction of current-policy trajectories. An augmented importance estimator uses a frozen outcome predictor to reduce residual variance, then takes exactly one policy update and consumes the data. This does not reuse rollout FLOPs: its plausible benefit is lower residual-gradient variance per total generation and scoring cost when the alternative proposal reaches informative regions. Current and stale actors may generate in parallel while the learner remains frozen. Learner–actor asynchrony is a separate non-theorem practical variant, not a claimed benefit of the core algorithm.

Our contributions are deliberately scoped:

- We define the target joint trajectory law, including the prompt and decoding distributions, and derive a stratified adaptive-mixture augmented policy-gradient estimator with bounded exact density ratio.
- We state sufficient conditions for conditional unbiasedness and the precise two-route bias identity. The result excludes ordinary replay, multiple epochs, clipping, and group standardization.
- We couple the estimator to an explicit expected-token constraint and provide logging and diagnostics that can falsify its assumptions.

[1]Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.

- We specify a frozen-policy estimator audit and a one-update-per-batch matched-budget study across model families and verifier domains.

Evidence status. No downstream or efficiency result is asserted in this draft. The theorem concerns one raw gradient estimate at a frozen policy; it neither guarantees a better optimizer step nor policy improvement.

2. Related Work

On-policy and group-relative RL. Careful PPO reproduction shows that reward whitening, KL treatment, clipping, and response length are implementation-sensitive (Huang et al., 2024). RLOO gives a simple leave-one-out REINFORCE baseline (Ahmadian et al., 2024); GRPO standardizes rewards within prompt groups (Shao et al., 2024); Dr. GRPO questions group-standard-deviation and response-length normalization (Liu et al., 2025b); and DAPO adds dynamic sampling, asymmetric clipping, and token-level aggregation (Yu et al., 2025). GSPO uses a length-normalized geometric mean of token ratios with response-level clipping (Zheng et al., 2025). Recent analyses expose negative-gradient pathologies (Deng et al., 2025), U-statistic structure (Zhou et al., 2026), mean-versus-variance calibration and KL-regularized success-probability recurrences (Mroueh, 2026), and the interaction between clipping, entropy, and misaligned rewards (Chen et al., 2026). Our estimator intentionally omits group normalization and PPO clipping so its target expectation remains explicit.

Allocation, baselines, and length. Adaptive rollout allocation predicts prompt difficulty and assigns rollout counts under a fixed budget (Nguyen et al., 2026). Dynamic Pruning Policy Optimization (DPPO) prunes redundant group rollouts and rescales retained samples by importance weights to preserve the full-batch GRPO expectation (Zhu et al., 2026). Cross-prompt shrinkage baselines reduce variance while preserving a policy-gradient target (Zeng et al., 2026). Group-relative reward rescaling addresses length inflation (Li et al., 2026), Length-Unbiased Sequence Policy Optimization (LUSPO) rectifies the length bias of GSPO by scaling sequence objectives by response length to prevent length collapse (Liu et al., 2026), and VL Norm (ΔL Normalization) reweights dynamic-length loss aggregation toward a minimum-variance unbiased policy-loss target (He et al., 2025). We instead keep the sequence score unnormalized, represent length as a constrained cost, and adapt the stale-source prompt proposal; these components are not claimed to subsume those methods. Discounted Beta–Bernoulli reward estimation reuses historical reward statistics to reduce group-estimator error while explicitly accepting bias (Kim et al., 2026). It is therefore a required bias–variance comparator: reusing summary statistics is distinct from our density-corrected use of newly drawn stale-policy trajectories.

Off-policy stability and reward validity. TOPR treats positive examples with an SFT/log-ratio-like term and truncates importance weighting for negative examples (Le Roux et al., 2025); buffer-based and balanced policy optimization improve practical reuse (Wan et al., 2026; Xi et al., 2026). Such stabilized estimators may be preferable even when biased. Separately, spurious-reward controls show that benchmark gains need not demonstrate learning from the intended verifier (Shao et al., 2026). The protocol includes those controls and treats exact statistical assumptions, not empirical stability alone, as the object of the first study. Direct replay studies optimize the generation–staleness–diversity trade-off (Arnal et al., 2026), while POPO prioritizes effective recent groups and uses decoupled off-policy correction (Mao et al., 2026). Both are stronger practical comparators than naive reuse. Their empirical utility does not imply the conditional unbiasedness claimed for our newly sampled, single-use design. AR3PO reuses earlier correct responses either through reward information only or with token probabilities recomputed under the current policy (Zhang et al., 2025). Recomputation changes the ratio but not the dependence of reused data on the current policy; Appendix A.4 gives an exact counterexample. Asynchronous RLHF separates generation from learning on lagged policies (Noukhovitch et al., 2025); it motivates the stale-actor systems layout but does not provide our conditional estimator identity. VCPO instead diagnoses ESS collapse under asynchronous policy lag and adapts the learning rate while using an off-policy variance-reducing baseline (Huang et al., 2026). It is a strong stability/throughput comparator, but changing the transformed optimizer step is distinct from auditing an unbiased raw estimator at one frozen policy. Statistically, the design is a restricted stratified multiple-importance sampler with augmentation. Classical work optimizes mixture allocation and control variates jointly (He & Owen, 2014); our pilot deliberately studies a small fixed grid rather than claiming a new general MIS principle or optimal allocation.

PrefixRL and PROS instead reuse an old prefix while generating a new continuation (Setlur et al., 2026; Huang & Wan, 2026); they change the prompt or conditional objective and avoid full-trajectory ratios. They are practical compute comparators, not instances of Equation (8). At the systems layer, HybridFlow decouples RL control and distributed computation (Sheng et al., 2025); our implementation follows that separation so rollout, cross-policy scoring, and update costs remain individually measurable. In its exact-acceptance setting, SPEC-RL uses old trajectories as speculative drafts while preserving the current-policy rollout law; its lenient setting does not have that exact claim (Liu et al., 2025a). MinPRO instead replaces exact causal-prefix ratios by a sta-

bilized minimum-ratio surrogate under policy lag (Lei et al., 2026). CTPO uses the cumulative product of per-token ratios. Under a token-level policy-gradient objective, this is an unbiased prefix correction with lower variance than the full-sequence ratio (Zhang et al., 2026), while Proximal KL combines a single current-token ratio with future KL credit so the expected update vanishes at the exact proximal optimizer under supported stale behavior mixtures without multiplying full-sequence ratios (Bai et al., 2026). Our estimand is the sequence-level score in Equation (4), so the exact weight is the joint mixture ratio of Equation (7), not a per-position product or a KL-regularized stationary-point surrogate. Score centering adds a correction term that cancels persistent drift between inference and training engines (Marek & Ryabinin, 2026). That mismatch is the practical form of our base-versus-decoder fault. We exclude it by requiring sampler–scorer conformance rather than correcting it, so score centering is a practical variant outside the theorem. VESPO reshapes sequence-level importance weights through a variance-constrained variational kernel without length normalization (Shen et al., 2026), while GGPO replaces learner-to-behavior ratios with a log-mean-exp contrast of KL-adjusted group rewards (Zhang & Zhang, 2026). Second-Moment Trust Policy Optimization (M2PO) constrains the batch second moment of importance weights to mask only extreme high-entropy token outliers under severe staleness (Zheng et al., 2026), and Coupled Off-Policy Correction (COPC) pairs token-level ratio masking with clipped-ratio TD-residual weighting to address advantage staleness (Hu et al., 2026). All four trade exact target-policy gradient unbiasedness for stability under severe staleness; our question is instead whether a defensive mixture with newly drawn stale trajectories reduces variance while keeping the exact one-step expectation. These methods target complementary systems and biased-stability trade-offs.

3. Formal Setting

Let $Z = (X, Y)$ contain prompt X and a complete generated sequence Y, including EOS or a declared truncation event. The target prompt distribution is D, a versioned finite prompt multiset with known masses $d(x)$, and the exact rollout distribution is $\pi_\theta(y \mid x)$. Thus

$$P_\theta(z) = d(x)\pi_\theta(y \mid x). \tag{1}$$

The rollout decoder is part of π_θ. Temperature, truncation, and any rejection rule must be represented by its normalized sampling probability; raw base-model logits are not automatically the rollout density. The theorem-facing implementation uses temperature-softmax sampling without top-k, top-p, repetition penalties, minimum-length masks, or forced tokens, with fixed maximum length and EOS included in the outcome. The allowed vocabulary $\mathcal{V}_\ell$ is a deterministic, parameter-independent function of the tokenizer and prefix. For temperature T_d the logged per-token mass is

$$p_\theta(y_\ell \mid y_{<\ell}, x) = \frac{\mathbf{1}\{y_\ell \in \mathcal{V}_\ell\} \exp(z_{\theta,\ell,y_\ell}/T_d)}{\sum_{v\in\mathcal{V}_\ell} \exp(z_{\theta,\ell,v}/T_d)}. \tag{2}$$

The sequence mass is the product through EOS; no-EOS sequences at the fixed limit carry their product mass and a deterministic truncation status. An empirical conformance test for both current and stale sampler–scorer pairs must pass before estimator audits.

The verifier deterministically maps every complete Z, including parse, timeout, sandbox-error, and truncation statuses, to bounded reward $R(Z)$; no outcome is retried or dropped. Interactive or stochastic environments are outside the core formulation. Generation incurs $C(Y) = |Y|$ response tokens. For fixed expected-response budget κ, we optimize the Lagrangian

$$\mathcal{L}(\theta, \lambda) = \mathbb{E}_{P_\theta}[R] - \lambda\{\mathbb{E}_{P_\theta}[C] - \kappa\}, \quad \lambda \geq 0. \tag{3}$$

The constrained problem is $\max_\theta \min_{\lambda\geq 0} \mathcal{L}(\theta, \lambda)$ with $\mathbb{E}_{P_\theta}[C] \leq \kappa$. The stochastic dual update below is a constraint controller, not a finite-sample feasibility guarantee. Define $U_\lambda = R - \lambda C$ and the raw sequence score

$$\begin{aligned} s_\theta(Z) &= \nabla_\vartheta \log \pi_\vartheta^{\text{decoder}}(Y \mid X)\big|_{\vartheta=\theta} \\ &= \sum_{\ell=1}^{|Y|} \nabla_\vartheta \log \pi_\vartheta^{\text{decoder}}(Y_\ell \mid X, Y_{<\ell})\big|_{\vartheta=\theta}. \end{aligned} \tag{4}$$

With $g_{\theta,\ell,v} = \nabla_\vartheta z_{\vartheta,\ell,v}|_{\vartheta=\theta}$, differentiating Equation (2) gives each step score $s_{\theta,\ell} = T_d^{-1}\big(g_{\theta,\ell,Y_\ell} - \sum_{v\in\mathcal{V}_\ell} p_\theta(v \mid Y_{<\ell}, X)\, g_{\theta,\ell,v}\big)$, which both scales by T_d^{-1} and centers token-logit Jacobians under the rollout decoder rather than the unscaled base softmax. We do not divide Equation (4) by realized length. Under standard score-function conditions, $\nabla_\theta \mathcal{L} = \mathbb{E}_{P_\theta}[U_\lambda s_\theta]$.

4. Method

Condition on the pre-update history $\mathcal{F}_t$. The current policy θ_t, dual variable, nuisance models, and all sampling distributions are frozen before trajectories are drawn. A logged adaptive prompt distribution Q_t and a stale behavior policy μ_t define

$$G_t(z) = q_t(x)\mu_t(y \mid x). \tag{5}$$

G_t is generated only after all iteration-t choices are frozen. A queue whose realized contents can affect θ_t, q_t, α, or nuisance models is not conditionally fresh and does not satisfy the theorem, even if its trajectories have never been optimized on.

For fixed $\alpha \in (0, 1]$, define the joint mixture

$$M_t(z) = \alpha P_{\theta_t}(z) + (1 - \alpha)G_t(z) \tag{6}$$

and exact detached weight

$$w_t(z) = \frac{P_{\theta_t}(z)}{M_t(z)} \le \frac{1}{\alpha}. \tag{7}$$

The prompt density is part of Equation (7). A conditional policy ratio is valid only when $Q_t = D$. Defensive mixtures and augmented importance estimators are established statistical tools (Owen & Zhou, 2000; Dudík et al., 2011); our contribution is their restricted, auditable instantiation for fresh stale-actor RLVR, not the general identities themselves.

4.1. Learning Objective

Let $h_t^R(z)$ and $h_t^C(z)$ be frozen action-conditional predictors of reward and length, and set $h_t = h_t^R - \lambda_t h_t^C$. This separation allows the dual variable to change without retraining the two predictors. Let $b_t(x)$ be a frozen prompt-only baseline. In stale-source conditions, fix integers $n_P, n_G \ge 1$ and mixture mass $\alpha \in (0,1)$ independently before sampling; write $\rho = n_P/(n_P+n_G)$ for the allocation fraction. Draw $Z_i^P \sim P_{\theta_t}$ and $Z_j^G \sim G_t$. For $\alpha < 1$, deterministic stratification gives

$$\widehat{g}_t = \frac{1}{n_P}\sum_{i=1}^{n_P}\Big[(h_i - b_i)s_i + \alpha w_i(U_i - h_i)s_i\Big] + \frac{1-\alpha}{n_G}\sum_{j=1}^{n_G} w_j(U_j - h_j)s_j. \tag{8}$$

The first line uses current-policy samples and the second uses stale-source samples. All w, h, b are stop-gradient quantities. The $\alpha = 1$ limit omits the stale-source sum and sets $n_G = 0$; it reduces exactly to ordinary on-policy $(U-b)s$. The outcome predictor is critic-like infrastructure; we do not call the method critic-free. The exploratory pilot restricts $\rho = \alpha$ to keep 64 total trajectories; the identity itself does not.

For conditionally IID trajectories within each of two independent strata define $A = (h-b)s + \alpha w(U-h)s$ under P and $B = w(U-h)s$ under G. Conditional on the frozen history,

$$\mathrm{Cov}(\widehat{g}_t) = \frac{\mathrm{Cov}_P(A)}{n_P} + \frac{(1-\alpha)^2\,\mathrm{Cov}_G(B)}{n_G}. \tag{9}$$

This expression includes covariance between direct and residual terms inside A. It gives no universal advantage over on-policy sampling; the frozen-policy audit chooses α from a preregistered grid using projected MSE times measured unit cost, then freezes it for confirmation. Nuisance fitting, α selection, and confirmation use three disjoint prompt partitions; the projection seed, dimension, reference sample, and confirmation replications are frozen before selection. At the artifact boundary, each successful current row stores projected A, each stale row stores projected B, and each reference row stores projected $U_\lambda s$. The aggregator must reconstruct $\overline{A} + (1-\alpha)\overline{B}$; a generic field called "gradient" is insufficient. Every attempted row also retains allocated accelerator-seconds, including failed and reference draws. Any infrastructure failure blocks estimator-claim eligibility rather than allowing a success-conditioned MSE. Projected terms are never trusted as declared. A raw-field validator recomputes each prompt's SHA-256(revision‖problem_id) hash rank and its exact partition boundary into D_{fit}, D_{select}, or D_{confirm}, recomputes the adaptive proposal $q_t(x) = \epsilon d(x) + (1-\epsilon)\overline{q}_t(x)$ from frozen prior-iteration residual second-moment and cost predictions (including the $Z_t^q = 0 \Rightarrow \overline{q}_t = D$ fallback), recomputes per-step masked temperature-softmax token log-probabilities from raw allowed-vocabulary step logits under T_d (requiring their sum to match both sequence log-probabilities), recomputes each step's decoder score vector $s_{\theta,\ell} = T_d^{-1}(g_{\theta,\ell,Y_\ell} - \sum_{v\in\mathcal{V}_\ell} p_\theta(v \mid Y_{<\ell}, X) g_{\theta,\ell,v})$ from per-token projected logit Jacobians and sums those vectors without length normalization to reproduce $s_\theta(Z)$ in Equation (4), verifies the deterministic verifier status-to-reward mapping (including truncation status consistency), checks finite-vocabulary sequence-tree normalization, sampler–scorer frequency conformance, and the policy-to-behavior KL stratum bound, recomputes w in log space from frozen header prompt masses and both sequence log-probabilities (together with per-cell, per-source maximum weights, $\max(\alpha w) \le 1$, and Kish effective sample sizes $\mathrm{ESS} = (\sum_i w_i)^2/\sum_i w_i^2$), and then reconstructs U, h, and A, B, or $U_\lambda s$ from rewards, lengths, the frozen λ, nuisance predictions, and projected scores. The projected ledger must match row for row. The same validator reconstructs K_{gen}, K_{ver}, K_{score}, and allocated accelerator-seconds across candidate, reference, and aborted attempts, and rejects rows whose generator or scorer is not the frozen policy pair under the rollout decoder (including stealth base-logit scoring that spoofs T_d, base-softmax score centering, or omitted T_d^{-1} score scaling), any token outside $\mathcal{V}_\ell$ or premature EOS, a length-normalized score vector, a reward that disagrees with its verifier status, a tampered weight ESS or bound, any reused trajectory, nuisance predictions from an unfrozen model or one trained on audited prompts, an omitted or spurious added-scoring pass, or an action-dependent baseline (Appendix A.4). It fails closed until the policy tensors are frozen. Before these partitions are formed, every prompt is tied to a pinned source revision, export-byte hash, stable upstream identifier, raw-row hash, and versioned normalized-content hash. The downstream partitioner assigns every retained duplicate cluster wholly to fit, selection, or confirmation; any later admitted hidden evaluator is a separate registered role. Semantic overlap with such evaluators is a blocking audit rather than being inferred from normalized-hash

inequality. A hash-bound preflight rejects named PII/secret patterns and retained character-5-gram near-duplicates, but this lexical heuristic does not replace reviewed multilingual, paraphrase, or hidden-evaluator leakage analysis. Above the exhaustive limit, only a reviewed exact engine whose hash-bound shards cover every cross-record field segment can lift the lexical blocker. The committed engine is unfrozen and indexes record pairs only, so it cannot clear a real multi-field export.

h_t^R, h_t^C, b_t, and q_t are trained only from data available before iteration t, then frozen. The sums below range over the complete finite prompt multiset, and prompts are sampled with replacement. A convenient adaptive proposal is

$$a_t(x) = d(x)\sqrt{\widehat{v}_t(x)/\widehat{c}_t(x)}, \tag{10}$$

$$Z_t^q = \sum_{x'} a_t(x'), \tag{11}$$

$$\overline{q}_t(x) = a_t(x)/Z_t^q, \tag{12}$$

$$q_t(x) = \epsilon d(x) + (1-\epsilon)\overline{q}_t(x). \tag{13}$$

where $\widehat{c}_t(x) > 0$ predicts generation cost and $\widehat{v}_t(x)$ uses prior-iteration random projections to predict stale-source residual-gradient second moment. Because q also changes the mixture weight and its covariance with the direct term, this square-root rule is a heuristic, not an optimum for Equation (8). Incorrect predictors may reduce efficiency but do not alter the density-route expectation when the logged distribution and ratio are exact. We impose $\widehat{v}_t(x) \geq v_{\min} > 0$ before normalization; equivalently, an implementation may set $\overline{q}_t = D$ whenever $Z_t^q = 0$.

4.2. Feedback and Credit Assignment

The main study uses deterministic programmatic verifiers: normalized answer checking for mathematics and isolated executable tests for code. Parser failures, timeouts, sandbox errors, and truncations are separate logged outcomes with predeclared scalar rewards rather than silently dropped samples. Hidden tests are disjoint from the authors' SFT, nuisance-model, and prompt-allocation data; unknown base-model pretraining contamination remains a limitation. Source and prompt draws are assigned before distributed generation. A valid sequence that reaches any declared verifier status is retained regardless of completion time. An infrastructure failure before a scored sequence exists aborts the complete batch, charges incurred cost, and performs no update; workers are never refilled selectively. This prevents selective refill but does not make completion ignorable. The theorem-facing audit therefore requires every preassigned draw to reach a scored outcome almost surely. If any infrastructure failure occurs, that replication is retained as an operational failure, never retried, and excluded from the estimator claim; H1 cannot pass from a success-conditioned subset.

Algorithm 1 One Adaptive-Mixture Update

1: Freeze $\theta_t, \lambda_t, h_t^R, h_t^C, b_t, q_t, \mu_t$, and α.
2: Draw n_P complete trajectories from $D\pi_{\theta_t}$.
3: Draw n_G new, single-use trajectories from $Q_t\mu_t$.
4: Log exact prompt, source, decoder, and sequence probabilities.
5: Evaluate verifier reward and token cost; form $U = R - \lambda_t C$.
6: Compute detached $w = P_{\theta_t}/M_t$ and Equation (8).
7: Finish accumulation at θ_t; call `optimizer.step()` once.
8: Consume every trajectory; no microbatch update or second epoch.
9: Update λ from fresh current-policy token costs.
10: Add consumed data only to future nuisance-model training.

Terminal U_λ multiplies the unnormalized sequence score in Equation (4); the theorem does not assign token-level causal credit. Reward shaping, group centering, group standardization, token-average losses, ratio clipping, and gradient clipping all change either the estimand or estimator and are excluded from the theorem-facing condition. They may be evaluated as clearly labeled practical variants. Shuffled, random, weakly related, and anti-correlated verifier controls test whether improvement follows the intended signal rather than pretrained behavior or evaluation leakage.

4.3. Algorithm and Complexity

Relative to on-policy REINFORCE, the method adds stale-actor generation, current and stale sequence log-probabilities under both policies, and nuisance-model inference. The cost ledger includes all generated tokens, verifier calls, likelihood evaluations, nuisance training, and policy forwards/backwards. Storage is linear in the single-use queue. A practical repeated-replay variant can reduce rollout cost, but is outside the theorem and is reported as biased.

5. Analysis

Theorem 5.1. *Condition on $\mathcal{F}_t$. Assume (i) all policies, prompt laws, nuisance models, counts, and α are fixed before sampling; (ii) Equation (1) is differentiable and differentiation and expectation may be interchanged; (iii) the deterministic reward does not directly depend on θ; (iv) b_t depends only on X; (v) current and stale samples are conditionally independent, newly drawn, and have not influenced θ_t; (vi) P_t and G_t have exactly evaluable masses on a common sample space, so M_t is computed exactly; (vii) every preassigned draw reaches a scored outcome almost surely; and (viii) required moments are finite. With exact Equation (7) and none of the clipping or normalization*

operations excluded above,

$$\mathbb{E}[\widehat{g}_t \mid \mathcal{F}_t] = \nabla_\theta \mathcal{L}(\theta_t, \lambda_t). \tag{14}$$

The proof expands the two strata:

$$\begin{aligned}\mathbb{E}[\widehat{g}_t] &= \mathbb{E}_P[(h-b)s] + \alpha\mathbb{E}_P[w(U-h)s] \\ &\quad + (1-\alpha)\mathbb{E}_G[w(U-h)s] \qquad (15)\\ &= \mathbb{E}_P[(h-b)s] + \mathbb{E}_M[w(U-h)s] \\ &= \mathbb{E}_P[(U-b)s] = \mathbb{E}_P[Us]. \qquad (16)\end{aligned}$$

The last equality is the prompt-baseline identity.

If $\widetilde{w}(z)$ is the same fixed, pointwise implemented weight in both strata and $m_0(z) = \mathbb{E}[U \mid Z = z]$, the population bias is

$$\mathbb{E}_M[(\widetilde{w} - w)(m_0 - h)s]. \tag{17}$$

This is a weight-misspecification identity only under frozen common weights. For source-specific fixed implementations it becomes

$$\alpha\mathbb{E}_P[(\widetilde{w}_P - w)(m_0 - h)s] + (1-\alpha)\mathbb{E}_G[(\widetilde{w}_G - w)(m_0 - h)s]. \tag{18}$$

Thus either exact density ratios or an exact action-conditional outcome model removes this bias. Low prediction error or calibration alone is insufficient, and focal-batch nuisance fitting, batch-normalized weights, and data-dependent weights are not covered. In deterministic RLVR, $m_0(z) = U(z)$, so an exact outcome model makes the stale residual vanish and leaves an on-policy direct estimator. We therefore describe Equation (8) as augmented importance sampling rather than relying on a broad "doubly robust replay" label.

Scope of the result. Theorem 5.1 applies before one update. After the policy changes, the current component of M_t is no longer the target policy, the $1/\alpha$ bound need not hold, and the same data influenced the updated policy. Multiple epochs, ratio recomputation, or small KL do not restore the theorem: in the exact four-cell law of Appendix A.4, reusing the batch that formed θ_t with a recomputed ratio remains biased although the post-update KL is below 0.01. Optimizer momentum and clipping may still be useful, but the theorem is about the raw estimator rather than the transformed displacement or monotonic improvement.

6. Experiments

Evaluation starts with a deterministic software Gate 0 on a finite trajectory law. It must exactly reproduce the target gradient, mixture normalization, weight bound, and analytic bias of each injected ratio error before Phase A. Phase A then establishes estimator behavior at a frozen language-model policy before Phase B makes training claims. The primary hypotheses are: **H1**, the selected exact adaptive-mixture estimator lowers projected-MSE–cost risk relative to an on-policy control on untouched confirmation draws; **H2**, any reduction translates to better held-out reward per generated token under one-update-per-batch training; and **H3**, the expected-response-length constraint produces a better reward–length frontier than implicit response-length normalization. Each can fail independently.

6.1. Experimental Setup

Frozen-policy audit. For each of two model families and both math and code verifiers, freeze a current policy, one stale policy, nuisance models, prompts, and decoders. For each target law generate two mutually independent large P-only reference estimates R_1, R_2, independent also of candidate estimate $\widehat{g}$, on adapter layers and projections frozen before outcomes. We estimate projected MSE by

$$\widehat{\mathrm{MSE}}_\times = (\widehat{g} - R_1)^\top (\widehat{g} - R_2). \tag{19}$$

Its expectation is projected MSE although one realization may be negative. Intervals jointly resample paired candidate replication IDs across cells and independently resample R_1 and R_2 trajectories, retaining shared-reference dependence. Every draw recomputes measured cost, nonnegative MSE–cost risk, and selected-minus-on-policy confirmation contrasts. Point estimates are independently recomputed from the hash-bound trajectory ledger: candidate stratum means, MSE, mean complete-replication cost, the selection argmin, and both selected and on-policy untouched confirmation cells must all reproduce. Reference-generation cost is reported separately rather than disappearing as a shared constant. Selection and confirmation use separate reference pairs under their respective laws. Nuisance and proposal models are trained on disjoint data. Repeated subsamples compare on-policy policy gradient, ordinary importance sampling, bounded mixture importance sampling, augmentation with several predictor qualities, and adaptive versus uniform Q_t. Endpoints are bias, variance, MSE, interval coverage, residual-weight ESS by source, and compute. Fault injection deliberately omits prompt ratios, uses base-model rather than decoder probabilities, fits the focal nuisance outcome, clips weights, and reuses data that formed the target policy. The committed finite-law Gate 0 executes all five injections by exact enumeration (Appendix A.4). Each fault must be detected, its positive control must reproduce the target to 10^{-12}, and its null-fault control must silence the detector. All five are also replayed through the raw trajectory reconstruction validator that gates every real train stage. These remain software checks on a generated law. The committed public-data configuration is a one-family mathematics pre-pilot for H1 only. It uses disjoint normalized laws D_{fit}, D_{select}, and D_{confirm}; selection MSE is not described as an estimate under the confirma-

tion or full-frame law. For $\alpha \in \{.25, .5, .75, 1\}$ it fixes $(n_P, n_G) \in \{(16, 48), (32, 32), (48, 16), (64, 0)\}$ and minimizes projected MSE times measured accelerator-seconds at common 64-trajectory and verifier-call counts. Because the unbiased cross-reference MSE can be negative, selection uses $\max(\widehat{\mathrm{MSE}}_{\times}, 0)$ times mean complete-replication accelerator-seconds while retaining the raw estimate; ties choose lower cost, then larger α. The audit freezes $\lambda = 0$ and counts EOS as a response token. Untouched confirmation includes the selected cell and the $\alpha = 1$ control under D_{confirm}. Tying mixture mass to allocation fraction is a pilot restriction, not a condition for unbiasedness; a confirmatory design must vary them separately. Code, second-family, H2, and H3 confirmatory runs require a later frozen configuration.

End-to-end training. Confirmatory training uses two open model families at small and medium scale, one exact-answer math suite and one sandboxed code suite. Prompts, hidden verifiers, maximum generation length, temperature, optimizer, and checkpoint selection rules are frozen before the main runs. Each collected batch produces one update and is never used again by the theorem-facing method. Staleness is stratified by measured policy KL, not checkpoint age.

Baselines are on-policy RLOO, Dr. GRPO, separately tuned DAPO and GSPO, TOPR, variance-informed prompt allocation (VIP), DBB reward estimation, PrefixRL, PROS, SPEC-RL, MinPRO, the ICML replay design, POPO, plain bounded-mixture importance sampling, augmented sampling with uniform Q, and the full adaptive proposal. All generated prefixes and complete sequences, verifier calls, likelihood passes, nuisance training, policy updates, and hyperparameter searches are charged. Each configuration receives the same token and verifier-call budget, while total allocated accelerator and CPU time is reported as a Pareto cost rather than assumed equal. We use five frozen estimator-pilot seeds. End-to-end development and confirmatory seed counts remain planned. Source-isolation controls set $\mu_t = \pi_t$ while adapting Q_t, and set $Q_t = D$ while varying policy staleness. Rollout counts are fixed before each batch. A batch is admitted only if its predeclared worst-case token and verifier-call reserve fits; the run stops at the last fully admitted batch and reports budget undershoot. Realized token, call, accelerator, and CPU costs are never converted into one unspecified scalar. Reward-versus-realized-token and reward-versus-accelerator curves are primary; endpoint contrasts are reported only at their largest common realized-cost checkpoint, with no interpolation across updates.

6.2. Main Results

Every value in Table 1 will aggregate independent runs with a predeclared interval and link to immutable trajectory manifests. The repeated-replay row is explicitly not covered by Theorem 5.1; it tests whether a biased engineering variant is nevertheless more useful. Estimator-claim eligibility records assumption and artifact checks; it is separate from the selected-versus-on-policy H1 estimate, interval, and result.

6.3. Ablations and Diagnostics

We ablate the on-policy anchor fraction α, augmentation, adaptive prompt proposal, prompt baseline, cross-iteration nuisance freezing, and length dual. A separate deliberate-violation study runs 1, 2, 4, and 8 epochs; uses fresh, previously consumed, and reward-selected stale data; and compares unclipped and clipped ratios. After each update, a large fresh on-policy batch estimates the resulting approximation error.

Required diagnostics include the numerical $1/\alpha$ bound; mixture normalization; residual-weight ESS by source; maximum weight and support failures; direct and residual gradient norms and cosine; outcome calibration by length, difficulty, and source; fresh versus augmented projected gradients; pre/post-update KL; entropy; sign-conditioned update mass; policy-to-behavior KL; dual trajectory; reward–token frontier; parser errors; and verifier disagreement. Reporting only reward curves is insufficient to test the method. The length study freezes $\kappa \in \{512, 1024, 2048\}$, $\lambda_0 = 0$, and dual step size 10^{-4}; last-iterate and final-20%-average frontiers are both reported. Replay comparators freeze admission, buffer capacity, reuse count, KL stratum, clipping, and update count, with original generation and rescoring charged.

6.4. Generalization and Safety

Generalization holds the estimator configuration fixed across model family, scale, prompt difficulty, context length, and verifier domain. Hidden verifier variants alter answer formatting, unit tests, parser implementation, and timeouts. Reward controls include shuffled labels, random rewards, weak heuristics, and anti-correlated signals. We measure train/hidden reward gaps, pass@1 and pass@k, response-length quantiles, invalid-output rates, memorized-answer indicators, and diversity. Safety claims are not inferred from verifier reward; the protocol will report separate capability and harmful-behavior audits as possible regressions.

Table 1. Planned primary results. Higher held-out reward is better; lower projected gradient MSE, MSE–cost risk, and generated tokens to threshold are better. Dashes mark measurements that have not yet been run.

Method	Projected MSE / risk	Held-out reward	Tokens to threshold	Mean response tokens
On-policy RLOO	–	–	–	–
Bounded mixture IS	–	–	–	–
Augmented mixture, uniform Q	–	–	–	–
Augmented mixture, adaptive Q	–	–	–	–
Practical repeated-replay variant	–	–	–	–

7. Limitations

The assumptions intentionally exclude the main appeal of replay: repeated use of old data. Single-use stale actors may add systems complexity without enough variance reduction to justify it. Exact sequence probabilities are expensive and fragile when production decoders use top-p, rejection, tool calls, or environment transitions. Adaptive prompt density must be logged exactly. Importance weights remain high-variance despite their bound, especially for small α and long horizons.

The outcome predictor is a critic by another name and may consume substantial compute. The two-route property in Equation (17) is practically asymmetric: behavior likelihoods are logged, whereas an exact sequence-conditional verifier predictor is unrealistic. Conditional gradient unbiasedness does not imply low MSE, stable Adam updates, monotonic improvement, or resistance to reward hacking. Finally, math and code evidence may not generalize to subjective human-feedback rewards or frontier models.

8. Conclusion

We separated statistically controlled stale generation from ordinary replay and specified an adaptive-mixture augmented policy gradient for expected-response-length-constrained RLVR. The estimator has a simple conditional expectation identity under strong, auditable restrictions, and fails to inherit that identity after data reuse or multiple updates. This clarity is useful only if the estimator actually reduces gradient error after all added compute is charged. The frozen-policy audit is therefore the next decisive step; downstream training should proceed only if that test succeeds.

Impact Statement

More sample-efficient RL could reduce the compute required to improve reasoning systems, but it can equally accelerate optimization toward flawed or harmful verifiers. Exact-looking tests can encode incomplete specifications, reward insecure code, or favor brittle answer formats. Adaptive allocation may also neglect rare prompts unless its support floor is enforced. We therefore require hidden-verifier variants, random and adversarial reward controls, minimum prompt support, full token and energy accounting, and separate safety audits. Stale trajectories and prompts may contain sensitive content and must follow the same retention and access controls as training data. The paper does not equate verifiability with correctness, harmlessness, or social value.

References

Ahmadian, A., Cremer, C., Gallé, M., Fadaee, M., Kreutzer, J., Pietquin, O., Üstün, A., and Hooker, S. Back to basics: Revisiting REINFORCE-style optimization for learning from human feedback in LLMs. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 12248–12267. Association for Computational Linguistics, 2024. doi: 10.18653/v1/2024.acl-long.662. URL https://aclanthology.org/2024.acl-long.662/.

Arnal, C., Cabannes, V., Cohen, T., Kempe, J., and Munos, R. Efficient RL training for LLMs with experience replay. In *International Conference on Machine Learning*, 2026. URL https://icml.cc/virtual/2026/poster/63133.

Bai, H., Yao, J., and Zhang, T. Enabling token-wise importance sampling in off-policy RLVR with proximal KL regularization, 2026. URL https://arxiv.org/abs/2610.18738.

Chen, P., Li, X., Li, Z., Yin, W., Chen, X., and Lin, T. Exploration vs exploitation: Rethinking RLVR through clipping, entropy, and spurious reward. In *International Conference on Learning Representations*, 2026. URL https://iclr.cc/virtual/2026/poster/10007078.

Deng, W., Ren, Y., Li, M., Sutherland, D. J., Li, X., and Thrampoulidis, C. On the effect of negative gradient in group relative deep reinforcement optimization. In *Advances in Neural Information Processing Systems*, volume 38, 2025. URL https://neurips.cc/virtual/2025/poster/120155.

Dudík, M., Langford, J., and Li, L. Doubly robust policy evaluation and learning. In *Proceedings of the 28th ICML*, pp. 1097–1104, 2011. URL https://arxiv.org/abs/1103.4601.

He, H. Y. and Owen, A. B. Optimal mixture weights in multiple importance sampling, 2014. URL https://arxiv.org/abs/1411.3954.

He, Z., Luo, X., Zhang, Y., Yang, Y., and Qiu, L. VL norm: Rethink loss aggregation in RLVR, 2025. URL https://arxiv.org/abs/2509.07558.

Hu, Z., Zhou, Z., Zhang, X., Liu, Y., Chen, C., Li, Y., Gu, Q., Feng, Y., Hao, H., and Qu, C. COPC: Coupled off-policy correction for asynchronous LLM reinforcement learning, 2026. URL https://arxiv.org/abs/2610.09597.

Huang, B. and Wan, X. PROS: Towards compute-efficient RLVR via rollout prefix reuse. In *International Conference on Learning Representations*, 2026. URL https://iclr.cc/virtual/2026/poster/10007630.

Huang, L., Zhang, Z., Hu, Q., Yang, S., and Han, S. Stable asynchrony: Variance-controlled off-policy RL for LLMs. In *International Conference on Machine Learning*, 2026. URL https://icml.cc/virtual/2026/poster/62012.

Huang, S., Noukhovitch, M., Hosseini, A., Rasul, K., Wang, W., and Tunstall, L. The N+ implementation details of RLHF with PPO: A case study on TL;DR summarization. In *First Conference on Language Modeling*, 2024. URL https://openreview.net/forum?id=kHO2ZTa8e3.

Kim, H., Ryu, S., Chu, G., Jang, D., and Yang, E. Discounted beta–bernoulli reward estimation for sample-efficient reinforcement learning with verifiable rewards. In *International Conference on Machine Learning*, 2026. URL https://icml.cc/virtual/2026/poster/64040.

Le Roux, N., Bellemare, M. G., Lebensold, J., Bergeron, A., Greaves, J., Fréchette, A., Pelletier, C., Thibodeau-Laufer, E., Toth, S., and Work, S. Tapered off-policy REINFORCE: Stable and efficient reinforcement learning for LLMs. In *Advances in Neural Information Processing Systems*, volume 38, 2025. URL https://neurips.cc/virtual/2025/poster/116762.

Lei, S., Cheng, Z., and Tao, D. A step back: Prefix importance ratio stabilizes policy optimization, 2026. URL https://arxiv.org/abs/2601.22718.

Li, Z., Lou, J., Dong, F., Fan, Z., Ren, M., Lin, H., Han, X., Zhang, D., Sun, L., Lu, Y., and Yu, X. Tackling length inflation without trade-offs: Group relative reward rescaling for reinforcement learning. In *International Conference on Machine Learning*, 2026. URL https://icml.cc/virtual/2026/poster/61369.

Liu, B., Wang, A., Min, Z., Yao, L., Zhang, H., Yang, L., Han, X., Li, P., Zeng, A., and Su, J. SPEC-RL: Accelerating on-policy reinforcement learning via speculative rollouts, 2025a. URL https://arxiv.org/abs/2509.23232.

Liu, F., Yin, Y., Shi, P., Yang, S., Zeng, Z., and Qiu, H. Length-unbiased sequence policy optimization: Revealing and controlling response length variation in RLVR, 2026. URL https://arxiv.org/abs/2602.05261.

Liu, Z., Chen, C., Li, W., Qi, P., Pang, T., Du, C., Lee, W. S., and Lin, M. Understanding R1-Zero-like training: A critical perspective. In *Second AI for Math Workshop at ICML*, 2025b. URL https://icml.cc/virtual/2025/52393.

Mao, Y., Qu, Y., Wang, Q., Zou, H., and Ji, X. RLVR without ineffective samples: Group prioritized off-policy optimization for LLM reasoning, 2026. URL https://arxiv.org/abs/2606.01281.

Marek, M. and Ryabinin, M. Score centering stabilizes off-policy reinforcement learning. *arXiv preprint arXiv:2609.20807*, 2026. URL https://arxiv.org/abs/2609.20807.

Mroueh, Y. Reinforcement learning with verifiable rewards: GRPO's loss, dynamics, and success amplification. In *International Conference on Machine Learning*, 2026. URL https://icml.cc/virtual/2026/poster/60548.

Nguyen, H. T., Nguyen, B., Ma, W., Zhao, Y., She, R., and Nguyen, V. A. Adaptive rollout allocation for online reinforcement learning with verifiable rewards. In *International Conference on Learning Representations*, 2026. URL https://iclr.cc/virtual/2026/poster/10008842.

Noukhovitch, M., Huang, S., Xhonneux, S., Hosseini, A., Agarwal, R., and Courville, A. Asynchronous RLHF: Faster and more efficient off-policy RL for language models. In *International Conference on Learning Representations*, 2025. URL https://openreview.net/forum?id=FhTAG591Ve.

Owen, A. B. and Zhou, Y. Safe and effective importance sampling. *Journal of the American Statistical Association*,

95(449):135–143, 2000. doi: 10.1080/01621459.2000.10473909.

Setlur, A., Wang, Z., Cohen, A., Rashidinejad, P., and Xie, S. M. Reuse your FLOPs: Scaling RL on hard problems by conditioning on very off-policy prefixes. In *ICLR Workshop on Scaling Post-training for LLMs*, 2026. URL https://iclr.cc/virtual/2026/10013427.

Shao, R., Li, S. S., Xin, R., Geng, S., Wang, Y., Oh, S., Du, S. S., Lambert, N., Min, S., Krishna, R., Tsvetkov, Y., Hajishirzi, H., Koh, P. W., and Zettlemoyer, L. Spurious rewards: Rethinking training signals in RLVR. In *International Conference on Machine Learning*, 2026. URL https://icml.cc/virtual/2026/poster/61082.

Shao, Z., Wang, P., Zhu, Q., Xu, R., Song, J., Bi, X., Zhang, H., Zhang, M., Li, Y. K., Wu, Y., and Guo, D. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models, 2024. URL https://arxiv.org/abs/2402.03300.

Shen, G., Zhao, C., Cheng, X., Huang, L., and Yu, X. VESPO: Variational sequence-level soft policy optimization for off-policy LLM training, 2026. URL https://arxiv.org/abs/2602.10693.

Sheng, G., Zhang, C., Ye, Z., Wu, X., Zhang, W., Zhang, R., Peng, Y., Lin, H., and Wu, C. HybridFlow: A flexible and efficient RLHF framework. In *Proceedings of the Twentieth European Conference on Computer Systems*, pp. 1279–1297, 2025. doi: 10.1145/3689031.3696075. URL https://doi.org/10.1145/3689031.3696075.

Wan, X., Wang, Y., Huang, W., and Sun, M. Buffer matters: Unleashing the power of off-policy reinforcement learning in large language model reasoning. In *International Conference on Learning Representations*, 2026. URL https://iclr.cc/virtual/2026/poster/10009473.

Williams, R. J. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8:229–256, 1992. doi: 10.1007/BF00992696.

Xi, Z., Guo, X., Nan, Y., Zhou, E., Shen, J., Chen, W., Liu, J., Huang, J., Zhang, Z., Guo, H., Deng, X., Lei, Z., Zheng, M., Wang, G., Zhang, S., Sun, P., Zheng, R., Yan, H., Gui, T., Zhang, Q., and Huang, X. BAPO: Stabilizing off-policy reinforcement learning for LLMs via balanced policy optimization with adaptive clipping. In *International Conference on Learning Representations*, 2026. URL https://iclr.cc/virtual/2026/poster/10007890.

Yu, Q., Zhang, Z., Zhu, R., Yuan, Y., Zuo, X., Yue, Y., Dai, W., Fan, T., Liu, G., Liu, J., Liu, L., Liu, X., Lin, H., Lin, Z., Ma, B., Sheng, G., Tong, Y., Zhang, C., Zhang, M., Zhang, R., Zhang, W., Zhu, H., Zhu, J., Chen, J., Chen, J., Wang, C., Yu, H., Song, Y., Wei, X., Zhou, H., Liu, J., Ma, W.-Y., Zhang, Y.-Q., Yan, L., Wu, Y., and Wang, M. DAPO: An open-source LLM reinforcement learning system at scale. In *Advances in Neural Information Processing Systems*, volume 38, 2025. URL https://neurips.cc/virtual/2025/poster/120129.

Zeng, G., Zhou, Z., Arora, D., and Zanette, A. Shrinking the variance: Shrinkage baselines for reinforcement learning with verifiable rewards. In *International Conference on Machine Learning*, 2026. URL https://icml.cc/virtual/2026/poster/64613.

Zhang, X. and Zhang, T. Gibbs-Gap policy optimization for off-policy RL with verifiable rewards, 2026. URL https://arxiv.org/abs/2610.20814.

Zhang, Y., Yao, W., Yu, C., Liu, Y., Yin, Q., Yin, B., Yun, H., and Li, L. Improving sampling efficiency in RLVR through adaptive rollout and response reuse, 2025. URL https://arxiv.org/abs/2509.25808.

Zhang, Y., Ye, C., Jin, S., Yu, C., Xiong, W., Sahu, S., and Jiang, N. Rethinking importance sampling in LLM policy optimization: A cumulative token perspective. *arXiv preprint arXiv:2605.07331*, 2026. URL https://arxiv.org/abs/2605.07331.

Zheng, C., Liu, S., Li, M., Chen, X.-H., Yu, B., Gao, C., Dang, K., Liu, Y., Men, R., Yang, A., Zhou, J., and Lin, J. Group sequence policy optimization, 2025. URL https://arxiv.org/abs/2507.18071.

Zheng, H., Zhao, J., and Chen, B. Prosperity before collapse: How far can off-policy RL reach with stale data on LLMs? In *International Conference on Learning Representations*, 2026. URL https://proceedings.iclr.cc/paper_files/paper/2026/file/8590b946e4f0c2b1262cb5b047924653-Paper-Conference.pdf.

Zhou, H., Ye, K., Xu, E., Zhu, J., Yang, Y., Gong, S., and Shi, C. Demystifying group relative policy optimization: Its policy gradient is a U-statistic, 2026. URL https://arxiv.org/abs/2603.01162.

Zhu, H., Ren, Y., Li, Y., Lin, M., Yang, L., Liu, X., Zhen, X., Liu, H., and Zhang, B. Unbiased dynamic pruning for efficient group-based policy optimization. In *International Conference on Machine Learning*, 2026. URL https://icml.cc/virtual/2026/poster/65323.

A. Algorithmic and Reproducibility Details

A.1. Dual Update

The expected-response-length dual uses only fresh target-policy trajectories:

$$\lambda_{t+1} = \left[\lambda_t + \eta_\lambda \left\{ \frac{1}{n_P} \sum_{i=1}^{n_P} C(Y_i^P) - \kappa \right\} \right]_+ . \tag{20}$$

κ is fixed before training. Policy and dual updates both estimate quantities at (θ_t, λ_t) and are committed after collection. Reward, response length, and violation are reported separately rather than only through U_λ. This projected stochastic controller does not guarantee that every iterate or the final policy is feasible. The constraint does not cover prompt tokens, likelihood passes, verifier cost, or total training compute; those belong to the experimental ledger.

A.2. Logging Contract

Each trajectory records prompt identifier and probability, source component, current and behavior checkpoint hashes, tokenizer, temperature, EOS and truncation rule, random seed, exact current and behavior sequence log-probabilities, reward components, parser status, generated tokens, verifier version, nuisance-model revision, use count, and policy-update identifier. The run aborts if a supposedly theorem-facing trajectory has use count above one or if its source probability cannot be reconstructed. For all candidate trajectories, including those in aborted batches, the ledger computes

$$K_{\text{gen}} = \sum_z C(z), \qquad K_{\text{ver}} = \sum_z \mathbf{1}\{\text{verifier called on } z\}, \tag{21}$$

$$K_{\text{score}} = \sum_z C(z)\, N_{\text{added score}}(z). \tag{22}$$

No cost disappears when a trajectory is excluded from the estimator.

A.3. Protocol Manifest

The repository runner `python3 scripts/run_experiment_pipeline.py --plan experiments/rl_pipeline.json` enumerates immutable prompt acquisition, verifier processing, policy and nuisance-model construction, and a frozen-policy H1 estimator pre-pilot. The full end-to-end H2/H3 plan is not executable from this configuration. The committed JSON is a stage and artifact contract tied to the hashed RL section of `experiments/pilot_preregistration.json`; site-local commands are required to execute it, and each executed stage records output hashes and separate token, verifier, accelerator, CPU, and wall-time costs. The public source registry pins the prompt source and base checkpoint only. The current and stale policy tensor hashes, tokenizer/decoder hashes, KL sampling law, and mathematics-verifier revision are still unfrozen blockers; this H1 pre-pilot cannot execute until a replacement configuration records them. A hidden code evaluator is outside this mathematics-only pre-pilot and must be registered by any later code study. Automated license metadata is only a preflight: real acquisition remains blocked on manual provenance, terms, privacy, and leakage review. Once approved, `scripts/materialize_public_data.py` verifies each site-local export hash, emits row-level provenance, deduplicates within roles, and quarantines normalized-content collisions across roles. The committed fixture exercises those rules without downloading prompts; all of its outputs are marked non-empirical. `scripts/run_content_gates.py` also binds the frozen `experiments/content_gates.json` contract to the materialization hashes and fails closed on retained named PII/secret patterns or lexical near-duplicates. Because the large-artifact MinHash-LSH candidate screen can miss pairs, it is inconclusive by itself. `scripts/validate_external_overlap_audit.py` admits a large-artifact result only after identifier-ordered, row-major upper-triangle shard ranges cover exactly $n(n-1)/2$ comparisons with no gap or overlap. The manifest binds the materialization, gate configuration, reviewed exact-engine source, runtime image, and review record; approximation, early termination, shard errors, and qualifying pairs are blocking. The validator separately checks every reported pair's ordinal, identifiers, roles, threshold, and score while retaining no matched text. A temporary disk-backed exact join also enumerates every pair sharing a 5-gram and rejects omissions; this is recall-complete for the positive frozen Jaccard thresholds because any qualifying pair must share a gram. Shard coverage is an engine-governance attestation, whereas readiness depends on reproducing the complete qualifying-pair set. Its committed engine hashes are null blockers, so only the generated contract self-test can currently pass. Even an exhaustive lexical pass neither clears a source

nor establishes semantic independence from a hidden evaluator. The external contract indexes record pairs rather than canonical field segments, so it cannot clear a multi-field export even after its engine fields are filled; a reviewed successor must cover every cross-record segment pair. `experiments/overlap_projection_contract.json` now enumerates every pilot source/role and requires separate model-input, training-target, protected-input, and protected-target slots where applicable. Its validator binds each inventory to source revision, configuration, split, exporter, canonical paths, and a review-artifact hash. Runtime enumeration requires every textual leaf to be projected or explicitly excluded, and training/protected fields are compared even within one record. Generated attestations cannot be relabeled as real. The schema audit covers the DAPO prompt and protected-answer classes, but the actual field inventory and review record remain null blockers. The companion `synthetic_commands.json` fixes a four-cell trajectory law, $\alpha = 1/2$, 32 draws per source, 2,000 replications, and seed 20260902. It enumerates the target and estimator expectations to machine precision, checks $\sum_z M(z)w(z) = 1$ and $w \le 1/\alpha$, and verifies the bias identity for omitted-prompt-ratio and weight-clipping errors. Gate 0 fails closed if any identity differs by more than 10^{-12}. This is an equation and software audit, not model-training evidence; its outputs cannot populate Table 1. `scripts/aggregate_rl_estimator.py` provides a second, artifact-level audit. Its generated two-dimensional ledger has hand-computed current, stale, and independent reference terms; the same code used for a real ledger must recover both selection-cell MSEs, the nonnegative-risk argmin, and selected plus on-policy confirmation MSEs. It jointly resamples complete paired candidate replication IDs across cells and independently resamples R_1 and R_2 trajectories within each target law. A domain-separated stream drives exact nearest-rank 2.5/97.5 percentile bounds for cell MSE, measured cost, risk, and selected-minus-control contrasts. The pilot refreezes a feasible shape of 64 projected coordinates, 250 candidate replications per cell, 2,048 trajectories per reference sample, and 2,000 bootstrap draws. NumPy 2.5.3 PCG64 emits exact multinomial weights in chunks of 32; projected means and squared norms avoid materializing resampled trajectory tensors, and the self-test executes the full selection-law shape. Tampered MSE, intervals, or cost counters and incomplete source strata must fail; infrastructure failure must block estimator-claim eligibility while still requiring the summary to reproduce the incurred candidate, reference, and aborted $K_{\mathrm{gen}}, K_{\mathrm{ver}}, K_{\mathrm{score}}$ and accelerator-seconds cost ledger. The fixture is marked non-empirical and cannot populate any result cell. Its input ledger and cost counters are no longer self-declared: `scripts/reconstruct_rl_trajectories.py` rebuilds every row and the complete $K_{\mathrm{gen}}, K_{\mathrm{ver}}, K_{\mathrm{score}}$ cost ledger from a raw header and raw trajectory ledger, and the runner requires exact agreement in both `train/trajectory_ledger.jsonl` and `train/costs.json`. The unreviewed field inventory, the null policy-pair, decoder, and verifier records, and the absence of a hash-bound sampler–scorer conformance artifact keep the artifact-integrity gate closed.

A.4. Exact Gate 0 Fault Injections

`scripts/rl_gate0_fault_injections.py` uses the same four-cell law as Gate 0: two prompts, a binary response, the scalar decoder-logit shift θ with score $y - \pi_x(\theta)$, $\alpha = 1/2$, and the frozen h and prompt baseline. Every expectation is an exhaustive sum over the joint sample space. The runner executes it and checks its hash before any stage. *Base-versus-decoder scoring* generates the current law with decoder temperature $T_d = 0.7$ and scores it with temperature-one base logits in both w and M. The total-variation distance between the two current laws is 0.058. The exact decoder route reproduces the target, whereas the base-scored estimator has bias 1.43×10^{-3}. That bias equals the common implemented-weight identity Equation (17) to 10^{-12}. *Focal nuisance fitting* refits h on the focal batch ($n_P = n_G = 2,256$ joint batches) as a pooled per-prompt outcome mean. Its bias is 4.93×10^{-3}. A frozen h and an h fitted on an independent prior current-policy batch (4,096 joint configurations) are both exact. *Consumed-trajectory reuse* forms θ_t by one REINFORCE step ($\eta = 1/2$) on two current-policy draws and then reuses those draws at θ_t. The target is the expected post-update gradient $\mathbb{E}[\nabla \mathcal{L}(\theta_t)]$. Fresh draws at θ_t reproduce it exactly. Reuse has bias 1.97×10^{-4} without a ratio and -2.53×10^{-4} with the recomputed ratio $P_{\theta_t}/P_{\theta_{t-1}}$, although the largest post-update KL is only 0.0074. The detection threshold is 10^{-6}. Setting $T_d = 1$, keeping h frozen in the focal route, or taking a zero step must each silence its detector, so a detector that always fires fails. These are exact properties of a constructed law, not estimates of language-model bias magnitudes. They show that the listed exclusions are necessary, not merely conservative.

Replay through raw reconstruction. The same law also defines a golden raw ledger. It has one row per cell and source for $\alpha = 1/2$, the $\alpha = 1$ control, and reference rows. Each row logs the prompt, per-step allowed-vocabulary token logits, EOS flags, per-token projected logit Jacobians, and step score vectors, both sequence log-probabilities, verifier status, reward, length, verifier call, added cross-policy scoring passes (1 for $\alpha < 1$ mixture candidates and 0 for $\alpha = 1$ and reference rows), nuisance predictions, and the unnormalized sequence score, while the header logs the hash-ranked prompt frame, the verifier status-to-reward table, prior-iteration residual second-moment and cost predictions, finite-vocabulary

sequence-tree conformance counts, the policy-to-behavior KL audit, and per-cell, per-source mixture-weight diagnostics ($\max w$, $\max(\alpha w)$, and Kish ESS). Reconstructed terms are unbiased for the target gradient by enumeration in all three groups to 10^{-12}, the adaptive proposal $q_t(x)$ and its $Z_t^q = 0$ fallback reproduce to 10^{-12}, the 2,048-prompt pilot frame reproduces its $1{,}024/512/512$ hash-rank partitions, and the reconstructed cost ledger recovers $K_{\text{gen}} = 24$, $K_{\text{ver}} = 12$, and $K_{\text{score}} = 16$ for candidates plus $K_{\text{gen}} = 8$, $K_{\text{ver}} = 4$, and $K_{\text{score}} = 0$ for reference rows; a separate aborted-attempt fixture verifies that failed draws retain their incurred token, verifier, and scoring costs without entering any gradient estimate. Thirty-one raw-ledger, prompt-frame, proposal, step-logit, logit-Jacobian, step-score, verifier-status, conformance, KL, weight-diagnostic, and cost-counter mutations fail, including a tampered prompt-selection hash, a wrong hash-rank prompt selected, a violated partition boundary, a step score centered with base softmax instead of the rollout decoder or missing T_d^{-1} scaling, a token-logit Jacobian shape mismatch, a length-normalized score instead of the unnormalized step sum, a verifier reward that disagrees with its status table, a truncated trajectory claiming a verified pass, a tampered source-specific weight ESS or maximum weight, mismatched adaptive-proposal moments, a violated zero-normalizer fallback, stealth base-logit scoring with a spoofed header temperature, a sampled token outside $\mathcal{V}_\ell$, a premature EOS token, an unnormalized conformance sequence tree, a policy-to-behavior KL above the stratum bound, dropping an aborted attempt's cost, or miscounting added scoring passes. They also match an independent producer written directly from the Gate 0 functions. All five faults are then replayed through the reconstruction. An omitted prompt ratio and clipping at $0.9 \max w$ change the producer's projected terms, with exact biases -1.71×10^{-2} and -1.61×10^{-3}, so the row-wise comparison rejects them. Base-logit scoring is rejected both when its recorded temperature differs from T_d and when it spoofs T_d while scoring sequence log-probabilities with unscaled step logits. When the validator is relaxed to accept base logits, the reconstructed bias equals the Gate 0 value 1.43×10^{-3} to 10^{-12}. Focal nuisance fitting is rejected either through overlap between training and audited prompts or through an unfrozen nuisance-model hash. Reuse is rejected through a nonzero prior use count or a generator hash from the previous policy, which covers both the plain and the ratio-recomputed variants. The null controls must pass: a proposal equal to D (including via $Z_t^q = 0$), a clip at $1/\alpha$, $T_d = 1$, disjoint nuisance training prompts, and fresh draws. Raw forward passes that produce the recorded step logits on the frozen checkpoints remain site-local hardware measurements.

A.5. Optional Prefix Gate

Prefix continuation is excluded from the core experiments. A later extension may draw a prefix $H \sim \mu_t$, choose a predictable $p_t(H) \in [p_{\min}, 1]$ before observing the suffix or reward, and include a completed stale-source residual with factor $I/p_t(H)$. The denominator remains the fixed number of candidate prefixes, not the number completed. Every discarded prefix is charged. Sequential gates require the product of all conditional continuation probabilities; group-relative estimators additionally require joint inclusion probabilities. This extension must be compared directly with PrefixRL and PROS (Setlur et al., 2026; Huang & Wan, 2026).

B. Additional Results and Failures

Table 2. Mandatory estimator-audit diagnostics; pending execution.

Diagnostic	Value
Projected bias relative to fresh target	–
Projected variance and MSE	–
Maximum mixture weight / $1/\alpha$	–
Joint effective sample size	–
Direct–residual gradient cosine	–
Prompt-density reconstruction error	–
Outcome calibration error	–
Added compute / on-policy compute	–

Complete learning curves, per-seed endpoints, theorem-violation results, reward-control trajectories, parser failures, and examples selected by a fixed random seed will be released whether they favor the method or not.